



Lanka Nippon Biztech Institute

Database Project

Advanced Database

Sathsarani Jayaweera

Name : J.M.Saubhagya

Index Number: UGC0122018

Batch :SE01

TABLE OF CONTENTS

TABLE OF CONTENTS

TABLE OF FIGURES

Chapter 1: INTRODUCTION

- 1.1 Introduction
- 1.2 Introduction of the system
- 1.3 Scope
- 1.4 Functionalities
- 1.5 Objectives
- 1.6 Summary

Chapter 2: DATABASE DESIGN

- 2.1 Introduction
- 2.2 Tables
- 2.3 Queries
- 2.4 Indexing
- 2.5 Functions
- 2.6 Procedures
- 2.7 Triggers
- 2.8 Transaction
- 2.9 Summary

Chapter 3: CONCLUSION

REFERENCES

TABLE OF FIGURES

Figure 1 2.2.1 User table

Figure 2 2.2.2 Posts table

Figure 3 2.2.3 Likes table

Figure 4 2.2.4 Notification table

Figure 5 2.3.1 Query 01

Figure 6 2.3.2 Query 02

Figure 7 2.3.3 Query 03

Figure 8 2.3.4 Query 04

Figure 9 2.3.5. Query 05

Figure 10 2.3.6. Query 06

Figure 11 2.3.7. Query 07

Figure 12 2.3.8. Query 08

Figure 13 2.3.9. Query 09

Figure 14 2.3.10. Query 10

Figure 15 2.4.1 Before indexing

Figure 16 2.4.1 Result

Figure 17 2.4.1 Create index

Figure 18 2.4.1 After create index

Figure 19 2.4.1 Cluster index

Figure 20 2.4.1 Before clustering in the category column

Figure 21 2.4.1 After clustering

Figure 22 2.4.2 Before indexing

Figure 23 2.4.2 Before indexing result

Figure 24 2.4.2 Create index

Figure 25 2.4.2 After create index using EXPLAIN query

Figure 26 2.4.2 EXPLAIN query result

Figure 27 2.4.2 create Cluster Index

Figure 28 2.4.2 before using Cluster index result

Figure 29 2.4.2 After using Cluster Index

Figure 30 2.4.3 Create Secondary Index

Figure 31 2.4.3 Secondary Index Result

Figure 32 2.5.1 Create function Most engaged age group

Figure 33 2.5.1 Created the function

Figure 34 2.5.1 Most engaged age group function's Result

Figure 35 2.5.2 Create Function Get comments by post

Figure 36 2.5.2 Created the function

Figure 37 2.5.2 Result in the get comments by post

Figure 38 2.5.3 Create a function call Count comments by user

Figure 39 2.5.3 Function created

Figure 40 2.5.3 Count comments by user function result

Figure 41 2.5.4 Create function Get comments by date range

Figure 42 2.5.4 Result in the Get comments by date range

Figure 43 2.6.1 Create delete comment table

Figure 44 2.6.1 Create procedure

Figure 45 Call the procedure

Figure 46 2.6.1 After delete the comments and call the delete comments table

Figure 47 2.6.2 Create procedure add user

Figure 48 2.6.2

Figure 49 2.6.2 Check when added new user in users table

Figure 50 2.6.3 Create procedure add comment

Figure 51 2.6.3 Added the comment in the comment table

Figure 52 2.7.1 Create function notify on new comment

Figure 53 2.7.1 Create trigger comment notification trigger

Figure 54 2.7.1 Insert the data in the comment table

Figure 55 2.7.1 Received message from the notification table

Figure 56 2.7.1 Insert the data in the comments table

Figure 57 2.7.2 Create function notify on new like

Figure 58 2.7.2 Create trigger like notification

Figure 59 2.7.2 Insert the likes table

Figure 60 2.7.2 Insert the value in the like table

Figure 61 2.7.2 Received notification in the notification table

Figure 62 2.8.1 Create transaction

Figure 63 2.8.1 Successfully BEGIN

Figure 64 2.8.1 Successfully insert the data in the likes table

Figure 65 2.8.1 Insert the notification table and COMMIT

Figure 66 2.8.1 Successful insert the likes table

Figure 67 2.8.1 Successful adding the notification in the notification table

Figure 68 2.8.2 Update the posts table

Figure 69 2.8.2 Result in the update posts table

Chapter 1: INTRODUCTION

1.1 Introduction

This social media platform database system aims to design scalable social media platform using PostgreSQL. The database will support user registration, posts, likes, notifications, comments and efficient data retrieval and query optimization. Indexing will be used to improve query performance, ensuring the platform fast and responsive.

1.2 Introduction the System

This project is a comprehensive social media platform design to facilitate user interactions through posts, likes, comments and notifications. The system includes five interconnected tables: **users**, **posts**, **comments**, **likes** and **notifications**. Users can create posts categorized by topics, engage with content through likes and comments, and receive real-time notifications about interactions on their posts. The platform also emphasizes data consistency and relational integrity using constraints and triggers. With features like user activity tracking, post categorization and engagement metrics this system provides a robust foundation for fostering meaningful online interactions.

1.3 Scope

The scope of this project is to develop a dynamic and interactive social media platform that allows to create engage with and manage content seamlessly. Key functionalities include:

1. User Management

- User registration and profile management

2. Content Management

- User can create, edit and categorize posts under relevant topics

3. Notification System

- Real-time notifications to inform users of interactions on their posts or other relevant activities.

4. Data Relationship

- Use of foreign key constraints and triggers to ensure data consistency and maintain relationships between entities.

This project provides a scalable framework for social interactions, catering to user needs for engagement and fostering an interactive digital community.

1.4 Functionalities

In this social media platform well designed PostgreSQL database can generate a variety of reports to help manage the platform, track user activity and analyze posts performance. Here are some key reports that would be valuable:

1. Get most engaged age group
2. Get comments for a specific post
3. Count comments by specific user
4. Get comments by date range

1.5 Objectives

1. Design and implementation a database for managing user, posts, likes, notifications and comments.
2. Use indexing for optimized the queries.
3. Retrieve posts by specific users.
4. Find posts with specific keywords.
5. Find users who liked posts with specific content.

1.6 Summary

In this chapter introduces social media platform focused on managing users, comments, notifications and likes efficiently. The chapter highlights the project's purpose, scope and core functions, providing a clear understanding of the system's objectives and benefits. In next chapter outlines the database schema and reports queries, transactions, indexing, procedures, triggers and functions.

Chapter 2: DATABASE DESIGN

2.1 Introduction

In the previous chapter, we discussed the system and its scope, what are its functionality, reports that can be made through this and objectives. In this chapter we will discuss tables in this social media project system and report queries, functions, indexing, procedures and transactions.

2.2 Tables

The database is a **social**. There are 5 tables which are users, notifications, posts, likes and comments. Here are the tables:

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username TEXT,  
    email TEXT,  
    password TEXT,  
    age int  
);
```

Figure 70 2.2.1 User table

The user table design to store information about users. Each user is identified by a unique Id. The table includes fields for the username, email, password and age.


```
CREATE TABLE posts (  
    id SERIAL PRIMARY KEY,  
    user_id INT NOT NULL,  
    category TEXT,  
    title TEXT,  
    content TEXT,  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

Figure 71 2.2.2 Posts table

In this table design to manage the posts made by users. Each post is identified by a unique Id. The table includes fields for category, title, content, created at and user id as foreign key.

```
CREATE TABLE likes (  
    id SERIAL PRIMARY KEY,  
    user_id INT NOT NULL,  
    post_id INT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE,  
    FOREIGN KEY (post_id) REFERENCES posts (id) ON DELETE CASCADE  
);
```

Figure 72 2.2.3 Likes table

In this table design to manage the likes made by users. Each like is identified by a unique Id. The table includes fields for category, title, content, created at and user id as foreign key.

```
CREATE TABLE notifications (  
    id SERIAL PRIMARY KEY,  
    user_id INT NOT NULL,  
    message TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Figure 73 2.2.4 Notification table

In this table design to manage the notification made by users. Each notification is identified by a unique Id. The table includes fields for message, created at and user id as foreign key.

2.3 Queries

In the social media platform, there are 5 tables. They are users, notifications, likes, posts and comments. In the below can see more queries that help to make decision on the system.

498
499
500
501
502
503
504
505
506
507
508
509

SELECT CASE

WHEN age BETWEEN 13 AND 17 THEN '13-17'

WHEN age BETWEEN 18 AND 24 THEN '18-24'

WHEN age BETWEEN 25 AND 34 THEN '25-34'

WHEN age BETWEEN 35 AND 44 THEN '35-44'

WHEN age BETWEEN 45 AND 54 THEN '45-54'

WHEN age BETWEEN 55 AND 64 THEN '55-64'

ELSE '65+'

END AS age_group, COUNT (l.id) AS like_count

FROM users u

JOIN likes l ON u.id = l.user_id

GROUP BY age_group ORDER BY like_count DESC;

Data Output

Messages

Notifications

SQL

	age_group text	like_count bigint
1	25-34	52
2	35-44	30
3	18-24	16
4	45-54	2

Figure 74 2.3.1 Query 01

This query counts the number of posts by users in different age groups.

```
SELECT EXTRACT (HOUR FROM l.created_at) AS hour_of_day,
COUNT (*) AS like_count
FROM likes l GROUP BY hour_of_day
ORDER BY like_count DESC;
```

	hour_of_day numeric	like_count bigint
1	19	8
2	14	8
3	17	8
4	18	8
5	11	8
6	13	8
7	16	8
8	10	8
9	15	8
10	12	8
11	9	8
12	20	6
13	8	4
14	21	2

Figure 75 2.3.2 Query 02

This query shows the most popular hour for posting, based on the created_at timestamp.

```

489
490 ✓ SELECT user_id, COUNT(*) AS post_count
491 FROM posts GROUP BY user_id
492 ORDER BY post_count DESC LIMIT 5;
493

```

Data Output Messages Notifications

	user_id integer	post_count bigint
1	11	6
2	19	6
3	4	6
4	8	6
5	3	6

Figure 76 2.3.3 Query 03

This query identifies the most active users based on the number of posts they have made.

```
SELECT post_id, COUNT (*) AS like_count
FROM likes GROUP BY post_id
ORDER BY like_count DESC LIMIT 5;
```

	post_id integer	like_count bigint
1	22	3
2	5	3
3	15	3
4	31	3
5	8	3

Figure 77 2.3.4 Query 04

This query shows the number of likes each post has received.

```
SELECT post_id, COUNT (*) AS comment_count
FROM comments GROUP BY post_id
ORDER BY comment_count DESC LIMIT 1;
```

	post_id integer	comment_count bigint
1	9	3

Figure 78 2.3.5. Query 05

This query identifies the post with the most comments.

```

SELECT CASE
  WHEN age BETWEEN 13 AND 17 THEN '13-17'
  WHEN age BETWEEN 18 AND 24 THEN '18-24'
  WHEN age BETWEEN 25 AND 34 THEN '25-34'
  WHEN age BETWEEN 35 AND 44 THEN '35-44'
  WHEN age BETWEEN 45 AND 54 THEN '45-54'
  WHEN age BETWEEN 55 AND 64 THEN '55-64'
  ELSE '65+'
END AS age_group, COUNT (l.id) AS like_count
FROM users u
JOIN likes l ON u.id = l.user_id
GROUP BY age_group ORDER BY like_count DESC;

```

	age_group text	like_count bigint
1	25-34	52
2	35-44	30
3	18-24	16
4	45-54	2

Figure 79 2.3.6. Query 06

This query shows the total number of likes made by each age group.

517	SELECT	category,	COUNT(*)	AS	post_count
518	FROM	posts	WHERE	category = 'Travel'	
519	GROUP BY	category;			
520					

Data Output	Messages	Notifications
+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	category text	post_count bigint
1	Travel	20

Figure 80 2.3.7. Query 07

This query shows how many posts are in specific category.

534	
535	SELECT
536	category, COUNT(*) AS total_posts
537	FROM posts GROUP BY category
538	ORDER BY total_posts DESC LIMIT 1;
539	

	category	total_posts
	text	bigint
1	Travel	12

Figure 81 2.3.8. Query 08

This query shows which post category has the most posts.


```

540 SELECT EXTRACT(HOUR FROM created_at) AS hour_of_day,
541 COUNT(*) AS post_count
542 FROM posts GROUP BY hour_of_day
543 ORDER BY post_count DESC LIMIT 1;
544

```

	hour_of_day numeric	post_count bigint
1	9	12

Figure 82 2.3.9. Query 09

This query identifies the hour of the day when users create the most posts.

```

545 SELECT p.category, COUNT(l.id) AS total_likes
546 FROM posts p
547 JOIN likes l ON p.id = l.post_id
548 GROUP BY p.category ORDER BY total_likes DESC
549 LIMIT 1;
550
551

```

	category text	total_likes bigint
1	Travel	11

Figure 83 2.3.10. Query 10

This query identifies the post category with the highest total likes, showing which type of content users engage with the most.

2.4 Indexing

This system uses secondary index and cluster index. A secondary index is an additional data structure created on a database table to allow for faster retrieval of records based on non-primary key column. A cluster index using for grouping the data.

01.

Before creating the index, using 'EXPLAIN ANALYZE' query to get the query plan and find the cost and first looked at what scan would come out like that.

```
-- Before create Index
```

```
EXPLAIN ANALYZE SELECT * FROM posts WHERE category = 'Romance';
```

Figure 84 2.4.1 Before indexing

Result:

```
680 -- Before create Index
```

```
681
```

```
682 EXPLAIN ANALYZE SELECT * FROM posts WHERE category = 'Romance';
```

```
683
```

Data Output Messages Notifications

SQL

	QUERY PLAN	
	text	
1	Seq Scan on posts (cost=0.00..3.25 rows=7 width=73) (actual time=0.013..0.023 rows=7 loops...	
2	Filter: (category = 'Romance'::text)	
3	Rows Removed by Filter: 93	
4	Planning Time: 0.999 ms	
5	Execution Time: 0.040 ms	

Figure 85 2.4.1 Result

Next create the index called 'posts_category'.

```
685
686 CREATE INDEX posts_category ON posts (category);
687
688
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 115 msec.

Figure 86 2.4.1 Create index

After creating the index, used the 'EXPLAIN ANALYZE' query again and got the new query plan. There I also found out what cost and which scan was used.

```
1108 -- After create Index
1109
1110 EXPLAIN ANALYZE SELECT * FROM posts WHERE category = 'Romance';
1111
```

Data Output Messages Notifications

QUERY PLAN

1	Bitmap Heap Scan on posts (cost=4.25..11.41 rows=13 width=78) (actual time=0.057..0.063 rows=13 loops=1)
2	Recheck Cond: (category = 'Romance'::text)
3	Heap Blocks: exact=3
4	-> Bitmap Index Scan on posts_category (cost=0.00..4.25 rows=13 width=0) (actual time=0.049..0.049 rows=13 loop...)
5	Index Cond: (category = 'Romance'::text)
6	Planning Time: 1.062 ms

Figure 87 2.4.1 After create index

Here I have used the Cluster index because there are several categories in the category list of the posts table and many posts are from different categories. I thought the Cluster index was good here for those reasons.

```
l116 -- Create CLUSTER index because I want to categorized posts
l117
l118 CLUSTER posts USING posts_category;
l119
```

Figure 88 2.4.1 Cluster index

Here is what the category column in the posts table looked like before using the Cluster index.

	id [PK] integer	user_id integer	content text	created_at timestamp without time zone	category text
500	278	15	The best wildlife safari experiences in Sri Lanka.	2025-01-10 15:00:00	Travel
501	511	2	Discovering hidden gems for solo travelers in Sri Lanka.	2025-02-13 09:00:00	Travel
502	512	4	How to stay motivated during a fitness journey in 2025.	2025-02-13 10:30:00	Health
503	513	3	The benefits of reading self-help books for personal growth in 2025.	2025-02-13 12:00:00	Books
504	514	5	How to organize a productive day working from home in 2025.	2025-02-13 13:30:00	Career
505	515	9	Top eco-friendly destinations in Sri Lanka to visit in 2025.	2025-02-13 15:00:00	Environment
506	516	7	How to create a balanced skincare routine in 2025 for glowing skin.	2025-02-13 16:30:00	Health
507	517	8	How to create a successful budget for your travels in 2025.	2025-02-13 18:00:00	Finance
508	518	10	The best Sri Lankan street food to try on your next visit.	2025-02-13 19:30:00	Food

Figure 89 2.4.1 Before clustering in the category column

Here after using the Cluster index, it shows how the category column in the posts table is located.

Data OutputMessagesNotifications

SQL

	id [PK] integer	user_id integer	content text	created_at timestamp without time zone	category text
42	362	13	How to create stunning artwork using digital tools in 2025.	2025-01-25 15:00:00	Art
43	156	3	How to write a captivating short story.	2024-12-21 18:30:00	Art
44	206	3	The best writing tips for beginners.	2024-12-30 18:30:00	Art
45	347	13	The top 10 art museums to visit in Sri Lanka in 2025.	2025-01-20 15:00:00	Art
46	228	9	How to choose the right camera for travel photography.	2025-01-03 13:40:00	Art
47	320	11	How to implement a green business model in Sri Lanka in 2025.	2025-01-16 16:30:00	Business
48	171	14	How to boost your entrepreneurial mindset for success.	2024-12-24 16:30:00	Business
49	489	13	How to start an online store in 2025 with minimal investment.	2025-02-09 18:00:00	Business
50	487	9	How to make your business more eco-friendly in 2025.	2025-02-09 15:00:00	Business
51	499	6	How to start a side hustle in 2025 while working full-time.	2025-02-11 12:00:00	Business
52	465	7	The top 10 books every entrepreneur should read in 2025.	2025-02-06 13:30:00	Business
53	461	11	The top 10 skills every entrepreneur should learn in 2025.	2025-02-05 18:00:00	Business
54	456	11	How to succeed as an entrepreneur in the digital age in 2025.	2025-02-05 10:30:00	Business
55	447	14	How to build a successful digital marketing strategy in 2025.	2025-02-03 18:00:00	Business
56	503	9	How to make the most of your digital marketing strategy in 2025.	2025-02-11 18:00:00	Business
57	442	7	How to market your small business online in 2025.	2025-02-03 10:30:00	Business

Total rows: 500 of 500Query complete 00:00:00.207Ln 1116, Col 60

Figure 90 2.4.1 After clustering

02.

Before creating the index, using 'EXPLAIN ANALYZE' query to get the query plan and find the cost and first looked at what scan would come out like that.

```
1470 -- Before create Index
1471
1472 EXPLAIN ANALYZE SELECT * FROM users WHERE age = 25;
1473
1474
```

Figure 91 2.4.2 Before indexing

Result:

QUERY PLAN	
1	Seq Scan on users (cost=0.00..15.71 rows=36 width=62) (actual time=0.021..0.053 rows=30 loops=...
2	Filter: (age = 25)
3	Rows Removed by Filter: 483
4	Planning Time: 0.075 ms
5	Execution Time: 0.600 ms

Total rows: 5 of 5 Query complete 00:00:00.178 Ln 1472, Col 1

Figure 92 2.4.2 Before indexing result

Next create the index called 'users_age'.

```
1473
1474 CREATE INDEX users_age ON users (age);
1475
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 106 msec.

Figure 93 2.4.2 Create index

After creating the index, used the 'EXPLAIN ANALYZE' query again and got the new query plan. There I also found out what cost and which scan was used.

```
1477 --After create Index
1478
1479 EXPLAIN ANALYZE SELECT * FROM users WHERE age = 25;
1480
```

Figure 94 2.4.2 After create index using EXPLAIN query

Result:

QUERY PLAN	
text	
1	Bitmap Heap Scan on users (cost=4.38..12.76 rows=30 width=62) (actual time=0.032..0.043 rows=30 loops=1)
2	Recheck Cond: (age = 25)
3	Heap Blocks: exact=6
4	-> Bitmap Index Scan on users_age (cost=0.00..4.38 rows=30 width=0) (actual time=0.025..0.025 rows=30 loops=1)
5	Index Cond: (age = 25)
6	Planning Time: 1.805 ms
7	Execution Time: 0.064 ms

Figure 95 2.4.2 EXPLAIN query result

Here I have used the Cluster index because there are several ages in the age list of the users table and many ages are from different age groups. I thought the Cluster index was good here for those reasons.

```
L481 --Create CLUSTER index
L482
L483 CLUSTER users USING users_age;
L484
```

Figure 96 2.4.2 create Cluster Index

Here is what the age column in the users table looked like before using the Cluster index.

522	17	rohan_de_silva	rohan.desilva@example.com	rohan_pass	43
523	8	manoj_perera	manoj@lanka.com	manoj321	45
524	673	dinesh_fernando	dinesh.fernando@lanka.com	dinesh2024!	33
525	674	rashmi_silva	rashmi.silva@xyz.com	rashmiPass456	29
526	675	anjali_perera	anjali.perera@lanka.com	anjaliSecure2023	26
527	676	kamal_wijesekera	kamal.wije@xyz.com	kamal1234	38
528	677	ishara_jayasinghe	ishara.jayasinghe@lanka.com	isharaPass@2024	31
Total rows: 528 of 528 Query complete 00:00:00.223 Ln 1504, Col 1					

Figure 97 2.4.2 before using Cluster index result

Here after using the Cluster index, it shows how the age column in the users table is located.

SQL					
	id	username	email	password	age
	[PK] integer	character varying (50)	character varying (100)	text	integer
1	479	lasith_gunawardana	lasith.guna@example.com	lasith@789	17
2	173	Vindya Hettiarachchi	vindya.hettiarachchi@example.com	vindya_pass	18
3	221	Madara Ekanayake	madara.ekanayake@example.com	ekanayake001	18
4	192	Harini Madushika	harini.madushika@example.com	madushika123	18
5	244	Malithi Jayasekara	malithi.jayasekara@example.com	jayasekara_123	18
6	161	Supuni Weerasinghe	supuni.weerasinghe@example.com	simplepass001	19
7	197	Kalana Gunasekara	kalana.gunasekara@example.com	kalana2024	19
8	7	kavi_silva	kavi_silva@lanka.com	kavi@123	19
9	236	Tharaka Hettiarachchi	tharaka.hettiarachchi@example.com	hettiarachchi_pass	19
Total rows: 513 of 513 Query complete 00:00:00.223 Ln 1360, Col 1					

Figure 98 2.4.2 After using Cluster Index

03.

This is a secondary index, this secondary index was created for the comments table, which was originally a table for which a cluster index was created. The secondary index is created for the user_id of the comment table.

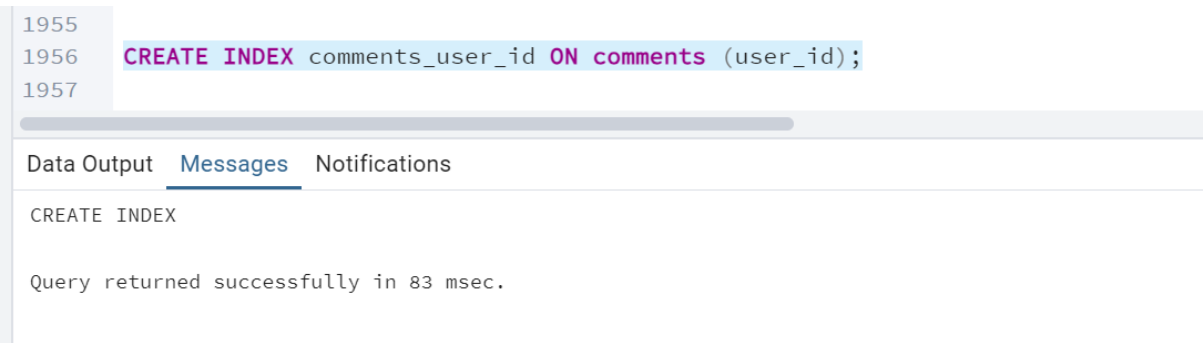


Figure 99 2.4.3 Create Secondary Index

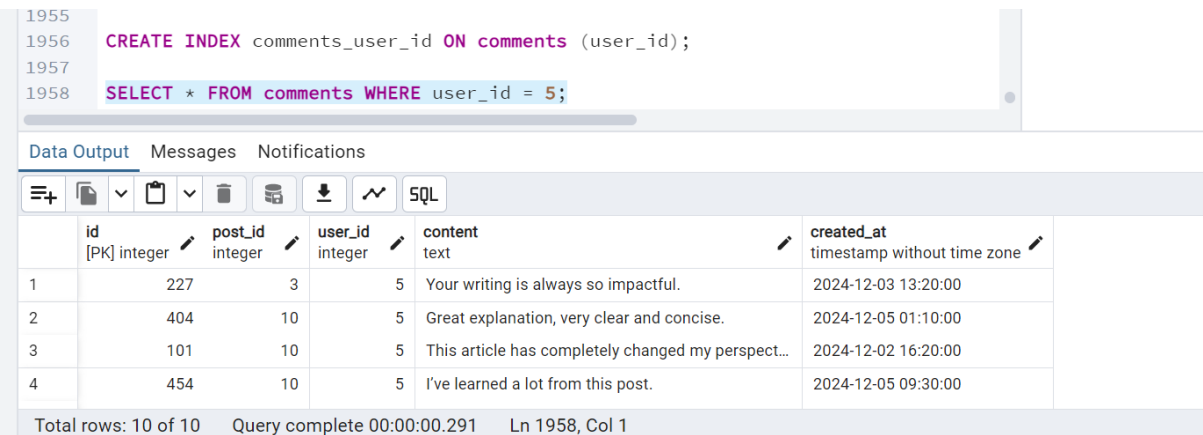


Figure 100 2.4.3 Secondary Index Result

2.5 Functions

In a function in PostgreSQL is a reusable of SQL code that performs a specific task and returns a value. Functions allows to automate tasks, reuse code and improve database performance. In their benefits are simplifies SQL code by reducing repetition, enhances modularity in database operations and can improve efficiency by executing code.

01. Get most engaged age group

Can use this function to find out which age group is most engaged with this social media database.

```
1961 CREATE OR REPLACE FUNCTION most_engaged_age_group()  
1962 RETURNS TABLE (  
1963     age_group TEXT,  
1964     total_comments INT  
1965 )  
1966 AS $$  
1967 BEGIN  
1968 RETURN QUERY  
1969 SELECT  
1970     CASE  
1971         WHEN age BETWEEN 13 AND 17 THEN 'Teen (13-17)'  
1972         WHEN age BETWEEN 18 AND 24 THEN 'Young Adult (18-24)'  
1973         WHEN age BETWEEN 25 AND 34 THEN 'Adult (25-34)'  
1974         WHEN age BETWEEN 35 AND 44 THEN 'Middle Age (35-44)'  
1975         WHEN age BETWEEN 45 AND 54 THEN 'Older Adult (45-54)'  
1976         ELSE 'Senior (55+)'  
1977     END AS age_group,  
1978     COUNT(comments.id) AS total_comments  
1979 FROM users  
1980 INNER JOIN comments ON user_id = comments.user_id  
1981 GROUP BY age_group  
1982 ORDER BY total_comments DESC  
1983 LIMIT 1;
```

Figure 101 2.5.1 Create function Most engaged age group

```
1961 CREATE OR REPLACE FUNCTION most_engaged_age_group()  
1962 RETURNS TABLE (  
1963     age_group TEXT,  
1964     total_comments INT  
1965 )  
1966 AS $$  
1967 BEGIN  
1968 RETURN QUERY  
1969 SELECT  
1970     CASE  
1971         WHEN age BETWEEN 13 AND 17 THEN 'Teen (13-17)'  
1972         WHEN age BETWEEN 18 AND 24 THEN 'Young Adult (18-24)'  
1973         WHEN age BETWEEN 25 AND 34 THEN 'Adult (25-34)'  
1974         WHEN age BETWEEN 35 AND 44 THEN 'Middle Age (35-44)'  
1975         WHEN age BETWEEN 45 AND 54 THEN 'Older Adult (45-54)'  
1976         ELSE 'Senior (55+)'  
1977     END AS age_group,
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 94 msec.

Figure 102 2.5.1 Created the function

The result of running this function is shown below.

2043

2044

2045

2046

```
SELECT * FROM most_engaged_age_group();
```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

	age_group text	total_comments bigint
1	Adult (25-34)	272

Figure 103 2.5.1 Most engaged age group function's Result

From this result, it can be seen that the age group that uses this social media database the most is the 25-35 Adult age group.

02. Get comments for a specific post

Here can get comments on special posts.

```
2072 ✓ CREATE OR REPLACE FUNCTION get_comments_by_post(post_id_param INT)
2073 RETURNS TABLE (
2074     comment_id INT,
2075     user_id INT,
2076     content TEXT,
2077     created_at TIMESTAMP
2078 )
2079 AS $$
2080 BEGIN
2081     RETURN QUERY
2082     SELECT c.id, c.user_id, c.content, c.created_at
2083     FROM comments c -- Alias
2084     WHERE c.post_id = post_id_param
2085     ORDER BY c.created_at DESC;
2086 END;
2087 $$ LANGUAGE plpgsql;
```

Figure 104 2.5.2 Create Function Get comments by post

```
2072 ✓ CREATE OR REPLACE FUNCTION get_comments_by_post(post_id_param INT)
2073 RETURNS TABLE (
2074     comment_id INT,
2075     user_id INT,
2076     content TEXT,
2077     created_at TIMESTAMP
2078 )
2079 AS $$
2080 BEGIN
2081     RETURN QUERY
2082     SELECT c.id, c.user_id, c.content, c.created_at
2083     FROM comments c -- Alias
2084     WHERE c.post_id = post_id_param
2085     ORDER BY c.created_at DESC;
2086 END;
2087 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

Figure 105 2.5.2 Created the function

2089
2090
2091

SELECT * FROM get_comments_by_post(12);

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

🔄

📥

📈

SQL

	comment_id integer	user_id integer	content text	created_at timestamp without time zone
1	456	7	This is exactly the insight I was looking for.	2024-12-05 09:50:00
2	406	7	This is such a useful piece of information.	2024-12-05 01:30:00
3	365	16	This is exactly what I needed to read today.	2024-12-04 18:40:00
4	343	44	This post is a must-read for everyone.	2024-12-04 15:00:00
5	282	44	This is such a well-organized post.	2024-12-03 22:30:00
6	229	9	I'm so glad you covered this topic.	2024-12-03 13:40:00
7	203	6	This gave me a lot to think about.	2024-12-03 09:20:00
8	168	39	This post has made me curious to learn more.	2024-12-03 03:30:00
9	119	23	Thanks for the effort in sharing this knowledge.	2024-12-02 19:20:00
10	53	5	Amazing perspective, I never thought of it this wa...	2024-12-02 08:20:00
11	3	5	Amazing work! Keep it up.	2024-12-01 08:20:00

Figure 106 2.5.2 Result in the get comments by post

According to the figure 42, the comments on a specific post with post id = 12 in the posts table, along with the user id of the user who commented, and the date the comment was posted, have been obtained.

03. Count comments by specific user

This function returns the total number of comments made by a specific user.

```
2093
2094 ✓ CREATE OR REPLACE FUNCTION count_comments_by_user(user_id_param INT)
2095 RETURNS INT AS $$
2096 DECLARE
2097     total_comments INT;
2098 ✓ BEGIN
2099     SELECT COUNT(*) INTO total_comments FROM comments
2100     WHERE userId = user_id_param;
2101     RETURN total_comments;
2102 END;
2103 $$ LANGUAGE plpgsql;
2104
```

Figure 107 2.5.3 Create a function call Count comments by user

```
2093
2094 ✓ CREATE OR REPLACE FUNCTION count_comments_by_user(user_id_param INT)
2095 RETURNS INT AS $$
2096 DECLARE
2097     total_comments INT;
2098 ✓ BEGIN
2099     SELECT COUNT(*) INTO total_comments FROM comments
2100     WHERE userId = user_id_param;
2101     RETURN total_comments;
2102 END;
2103 $$ LANGUAGE plpgsql;
2104
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 75 msec.

Figure 108 2.5.3 Function created

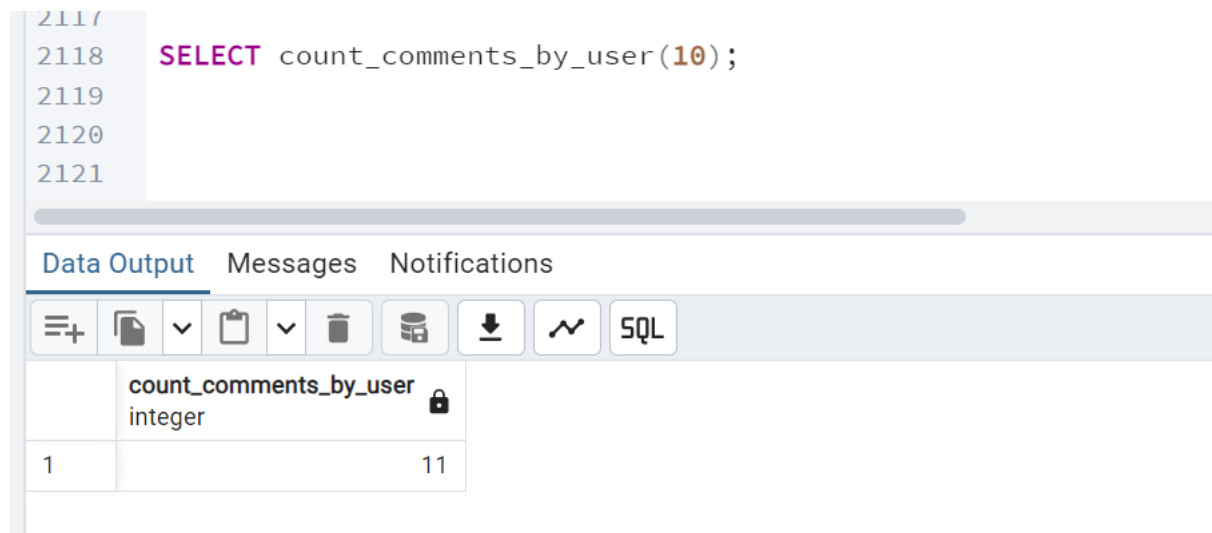


Figure 109 2.5.3 Count comments by user function result

As shown in Figure 45 above, a user with a user id of 10 has add 11 comments. This function returns the number of comments add by specific user.

04. Get comments by date range

This function retrieves all comments made within a specific date range.

```

2122 CREATE OR REPLACE FUNCTION get_comments_by_date_range(start_date TIMESTAMP, end_
2123 RETURNS TABLE (
2124     comment_id INT,
2125     user_id INT,
2126     post_id INT,
2127     content TEXT,
2128     created_at TIMESTAMP
2129 )
2130 AS $$
2131 BEGIN
2132     RETURN QUERY
2133     SELECT
2134         comments.id AS comment_id,
2135         comments.user_id,
2136         comments.post_id,
2137         comments.content,
2138         comments.created_at
2139     FROM comments
2140     WHERE comments.created_at BETWEEN start_date AND end_date
2141     ORDER BY comments.created_at;
2142 END;
2143 $$ LANGUAGE plpgsql;
2144

```

Figure 110 2.5.4 Create function Get comments by date range

2143

2146

2147

SELECT * FROM get_comments_by_date_range('2024-11-15 00:00:00', '2024-12-15 23:11:00')

Data OutputMessagesNotifications

SQL

	comment_id integer	user_id integer	post_id integer	content text	created_at timestamp without time zone
1	1	3	5	Great post! Really enjoyed reading this.	2024-12-01 08:00:00
2	2	7	8	Thanks for sharing this information!	2024-12-01 08:10:00
3	3	5	12	Amazing work! Keep it up.	2024-12-01 08:20:00
4	4	9	18	Can you elaborate more on this topic?	2024-12-01 08:30:00
5	5	4	22	This is so relatable! Thank you.	2024-12-01 08:40:00
6	6	8	25	What inspired you to write this?	2024-12-01 08:50:00
7	7	2	27	Loved the examples you provided!	2024-12-01 09:00:00
8	8	6	30	This post really made me think.	2024-12-01 09:10:00
9	9	10	34	Can you recommend resources on this topic?	2024-12-01 09:20:00
10	10	12	38	Awesome! Looking forward to more posts like this.	2024-12-01 09:30:00
11	11	15	41	This is exactly what I needed right now.	2024-12-01 09:40:00

Figure 111 2.5.4 Result in the Get comments by date range

This shows a description of the comments added to the comments table over a period of time, as shown in figure 47 above.

2.6 Procedures

A procedure in PostgreSQL is a reusable block of code that performs a specific task but does not return values. Unlike functions, procedures can execute transactions such as COMMIT and ROLLBACK making them useful for managing complex business logic that involves multiple steps or operations.

01. Delete comment

This procedure is creating ensure that when a user deletes a comment, the comment is not permanently lost but instead moved to separate table, **delete comments**, for record keeping. This approach is useful for maintaining a history of delete comments, complying with data relation policies, or debugging user actions. The procedure achieves this by inserting the delete comment's details into the delete comments table and then moving it from the comments table.

```
2147 -----Create new table name delete comment and truncate the delete comments table
2148
2149
2150 CREATE TABLE delete_comments(
2151     id SERIAL PRIMARY KEY,
2152     comment_id INT NOT NULL,
2153     user_id INT NOT NULL,
2154     post_id INT,
2155     content TEXT,
2156     created_at TIMESTAMP,
2157     delete_at TIMESTAMP DEFAULT NOW()
2158 );
2159
```

Figure 112 2.6.1 Create delete comment table

As shown in Figure 43, a new table called delete comments is created. This is because if a user deletes their comments, all the details are truncated.

```

2159
2160 -----Create procedure delete comments
2161
2162 ✓ CREATE OR REPLACE PROCEDURE delete_comment(comment_id INT) AS $$
2163 BEGIN
2164
2165 -----insert the delete comment in the delete table
2166
2167 INSERT INTO delete_comments (comment_id, user_id, post_id, content, created_at)
2168 SELECT id, user_id, post_id, content, created_at
2169 FROM comments WHERE id = comment_id;
2170
2171 ----Delete comment from original comment table
2172 DELETE FROM comments WHERE id = comment_id;
2173 END;
2174 $$ LANGUAGE plpgsql;
2175

```

Figure 113 2.6.1 Create procedure

Created the delete comment procedure.

```

2175
2176 CALL delete_comment(20);
2177

```

Figure 114 Call the procedure

Call the delete comment procedure.

2177
2178 SELECT * FROM delete_comments;
2179

Data Output Messages Notifications							
	id [PK] integer	comment_id integer	user_id integer	post_id integer	content text	created_at timestamp without time zone	delete_at timestamp without time
1	1	20	22	28	What's your opinion on related topics?	2024-12-01 11:10:00	2024-12-16 12:15:13.045

Figure 115 2.6.1 After delete the comments and call the delete comments table

Here, the comment with comment id = 20 is deleted from the original comments table and all of its details are truncated to the newly created delete comments table.

02. Add user

What happens in this procedure is that a new user is added to the database.

```
2185 CREATE OR REPLACE PROCEDURE add_user(  
2186     username TEXT,  
2187     email TEXT,  
2188     password TEXT,  
2189     age INT  
2190 )  
2191 AS $$  
2192 BEGIN  
2193     INSERT INTO users(username, email, password, age, created_at)  
2194     VALUES (username, email, password, age, NOW());  
2195 END;  
2196 $$ LANGUAGE plpgsql;  
2197
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 92 msec.

Figure 116 2.6.2 Create procedure add user

```
2197  
2198 CALL add_user('Matheesha Pathirana', 'mathessha@gmail.com', 'ma990', 25);  
2199  
2200
```

Data Output Messages Notifications

CALL

Query returned successfully in 76 msec.

Figure 117 2.6.2

Call the procedure add user.

2200 **SELECT * FROM users;**

Data Output Messages Notifications

SQL

	id [PK] integer	username character varying (50)	email character varying (100)	password text	age integer
520	271	pradeep_gunawardena	pradeep.guna@example.com	pradeep_456	42
521	52	tharaka_abeyasinghe	tharaka.abey@example.com	tharaka2023	42
522	142	nipuna_perera	nipuna.perera@lanka.com	nipuna123	42
523	126	dushan_silva	dushan.silva@lanka.com	dushanPass2023	42
524	363	roshan_fernando	roshan.fernando@example.com	roshan_007	42
525	299	pasindu_wijesinghe	pasindu.wije@example.com	pasinduPass456	42
526	33	niranjana_perera	niranjana.perera@lanka.com	niranjana_456	42
527	17	rohan_de_silva	rohan.desilva@example.com	rohan_pass	43
528	8	manoj_perera	manoj@lanka.com	manoj321	45
529	678	Matheesha Pathirana	mathessha@gmail.com	ma990	25

Total rows: 529 of 529 Query complete 00:00:00.259 Ln 2200, Col 1

Figure 118 2.6.2 Check when added new user in users table

03. Add comment

```
2204 CREATE OR REPLACE PROCEDURE add_comment(  
2205     user_id INT,  
2206     post_id INT,  
2207     content TEXT  
2208 )  
2209 AS $$  
2210 BEGIN  
2211     INSERT INTO comments (user_id, post_id, content, created_at)  
2212     VALUES (user_id, post_id, content, NOW());  
2213 END;  
2214 $$ LANGUAGE plpgsql;  
2215
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 126 msec.

Figure 119 2.6.3 Create procedure add comment

In this procedure, a new comment is added to the comment table through this procedure.

```
2216 CALL add_comment(2,10, 'This is amazing');
```

Data Output Messages Notifications

	id [PK] integer	post_id integer	user_id integer	content text	created_at timestamp without time zone
500	501	53	2	This is such an interesting perspective!	2024-12-11 08:00:00
501	502	54	3	I really appreciate the depth of this post!	2024-12-11 08:10:00
502	503	55	4	I've been thinking about this topic a lot lately, great timin...	2024-12-11 08:20:00
503	504	56	5	I can't wait to see more posts like this!	2024-12-11 08:30:00
504	505	57	6	This post answers a lot of questions I had, thank you!	2024-12-11 08:40:00
505	506	58	7	I love how you broke this down, very informative.	2024-12-11 08:50:00
506	507	59	8	This is one of the best posts I've read in a while!	2024-12-11 09:00:00
507	508	60	9	Your insights are always so helpful, keep it up!	2024-12-11 09:10:00
508	509	61	10	Great job on this post, it really made me think.	2024-12-11 09:20:00
509	510	62	11	I'm bookmarking this post for later, excellent content!	2024-12-11 09:30:00
510	511	10	2	This is amazing	2024-12-17 05:59:23.0939

Figure 120 2.6.3 Added the comment in the comment table

As shown in the image, the newly added comment has been added to the comments table through the procedure.

2.7 Triggers

A trigger in PostgreSQL is a database object that automatically executes a specific function in response to certain events on a table. These events can include INSERT, UPDATE or DELETE operations. Triggers help automate tasks, enforce business rules, maintain audit trails or ensure data consistency across related tables.

01. Notify user when a new comment is added to their post

This feature ensures are notified whenever someone comments on their post. A trigger is used to automatically add a notification to the notifications table when a new comment is added to the comments table. The function notify on new comment retrieves the post owner's user id from the posts table using the post id of the comment. It then creates a notification with the type comment and message, "Your post received a new comment". The trigger comment notification trigger activates this function after a new comment is inserted, ensuring notifications are sent without manual effort.

```
2376 CREATE OR REPLACE FUNCTION notify_on_new_comment() RETURNS TRIGGER
2377 AS $$
2378 BEGIN
2379     INSERT INTO notifications(user_id, message, created_at)
2380     VALUES (
2381         (SELECT user_id FROM posts WHERE id = NEW.post_id),
2382         'Your post received a new comment',
2383         NOW()
2384     );
2385     RETURN NEW;
2386 END;
2387 $$ LANGUAGE plpgsql;
```

Data Output

Messages

Notifications

CREATE FUNCTION

Query returned successfully in 98 msec.

Figure 121 2.7.1 Create function notify on new comment

```
2389 CREATE TRIGGER comment_notification
2390 AFTER INSERT ON comments
2391 FOR EACH ROW
2392 EXECUTE FUNCTION notify_on_new_comment();
2393
```

Figure 122 2.7.1 Create trigger comment notification trigger

```
INSERT INTO comments (user_id, post_id, content, created_at)
VALUES (2, 1, 'Great post!', NOW());
```

Figure 123 2.7.1 Insert the data in the comment table

Result:

102	102	5	Your post received a new comment	2024-12-18 05:43:52.996417
103	103	5	Your post received a new comment	2024-12-18 05:43:52.996417
Total rows: 103 of 103 Query complete 00:00:00.097 Ln 2397, Col 1				

Figure 124 2.7.1 Received message from the notification table

510	511	10	2	This is amazing	2024-12-17 00:00:00.000000
511	526	1	2	Great post!	2024-12-18 05:43:52.996417

Figure 125 2.7.1 Insert the data in the comments table

As shown in Figures 55 and 56 above, the data inserted into the comments table is inserted into the comments table, and a message has been sent to the notification table indicating that a new comment has been added to that post.

02. Notify when a post is liked

This trigger causes a notification to appear the notification table when a post is liked, saying 'your post received a like'.

```
2546 ✓ CREATE OR REPLACE FUNCTION notify_on_new_like() RETURNS TRIGGER AS $$  
2547 BEGIN  
2548     INSERT INTO notifications(user_id, message, created_at)  
2549     VALUES (  
2550         (SELECT user_id FROM posts WHERE id = NEW.post_id),  
2551         'Your post received a new like',  
2552         NOW()  
2553     );  
2554     RETURN NEW;  
2555 END;  
2556 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 95 msec.

Figure 126 2.7.2 Create function notify on new like

```
2534 ✓ CREATE TRIGGER like_notification  
2535 AFTER INSERT ON likes  
2536 FOR EACH ROW  
2537 EXECUTE FUNCTION notify_on_new_like();  
2538
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 95 msec.

Figure 127 2.7.2 Create trigger like notification

Result:

2539

▼

INSERT INTO likes (user_id, post_id, created_at)

2540

VALUES (10, 5, NOW());

Data Output

Messages

Notifications

INSERT 0 1

Figure 128 2.7.2 Insert the likes table

101	101	10	5	2024-12-18 06:21:49.800402
-----	-----	----	---	----------------------------

Figure 129 2.7.2 Insert the value in the like table

2397

SELECT * FROM notifications;

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	id [PK] integer	user_id integer	message text	created_at timestamp without time zone
96	96	50	Your direct message was read by the recipient.	2024-12-15 10:15:00
97	97	51	Your post received 25 reactions.	2024-12-15 11:00:00
98	98	52	You were mentioned in a group conversation.	2024-12-15 11:45:00
99	99	53	Your event RSVP has been confirmed.	2024-12-15 12:30:00
100	100	54	Your friend request was declined.	2024-12-15 13:15:00
101	101	55	Your account settings were restored to default.	2024-12-15 14:00:00
102	102	5	Your post received a new comment	2024-12-18 05:43:52.996417
103	103	5	Your post received a new comment	2024-12-18 05:43:52.996417
104	104	8	Your post received a new like	2024-12-18 06:21:49.800402
105	105	8	Your post received a new like	2024-12-21 13:51:59.21563

Figure 130 2.7.2 Received notification in the notification table

2.8. Transactions

A transaction in PostgreSQL is a sequence of one or more SQL operations performed as a single, logical unit of work. Transaction ensures data integrity and consistency by following the ACID properties.

01. Add a like and notify the post owner

In this transaction that when a user like a post, a record is added to the likes table and a notification is sent the owner of post.

```
BEGIN;  
  
--Insert a like into the like table  
  
INSERT INTO likes (user_id, post_id) VALUES (20, 25);  
  
-----Insert a notification for the post owner  
INSERT INTO notifications(user_id, message, created_at)  
VALUES (  
    (SELECT user_id FROM posts WHERE id = 25),  
    'Someone liked your post',  
    NOW()  
);  
  
COMMIT;
```

Figure 131 2.8.1 Create transaction

The screenshot shows a PostgreSQL client window with a query editor on the left and a results pane on the right. The query editor contains the SQL command `BEGIN;`. The results pane has three tabs: "Data Output", "Messages", and "Notifications". The "Messages" tab is selected and shows the message "Query returned successfully in 102 msec." Below the message, there is a table with one row and one column, containing the value "BEGIN".

BEGIN

Figure 132 2.8.1 Successfully BEGIN

```
2567 --Insert a like into the like table
2568
2569 INSERT INTO likes (user_id, post_id) VALUES (20, 25);
2570
```

Data Output Messages Notifications

INSERT 0 1

Query returned successfully in 96 msec.

Figure 133 2.8.1 Successfully insert the data in the likes table

```
2568
2569 INSERT INTO likes (user_id, post_id) VALUES (20, 25);
2570
2571 -----Insert a notification for the post owner
2572 ✓ INSERT INTO notifications(user_id, message, created_at)
2573 VALUES (
2574     (SELECT user_id FROM posts WHERE id = 25),
2575     'Someone liked your post',
2576     NOW()
2577 );
2578
2579 COMMIT;
2580
```

Data Output Messages Notifications

COMMIT

Query returned successfully in 91 msec.

Figure 134 2.8.1 Insert the notification table and COMMIT

Result:

100	100	20	25	2024-12-21 14:40:57.632278
104	104	20	25	2024-12-21 14:40:57.632278
Total rows: 104 of 104 Query complete 00:00:00.243 Ln 2574, Col 45				

Figure 135 2.8.1 Successful insert the likes table

108	108	10	Someone liked your post	2024-12-21 14:40:57.632278
Total rows: 108 of 108 Query complete 00:00:00.190 Ln 2583, Col 2				

Figure 136 2.8.1 Successful adding the notification in the notification table

02. Update the posts table

This query a basic transactional update operation. It begins by updating the posts table, modifying the title, content and category of specific post identified by its id. The transaction ensure data consistency where all changes are committed as a single unit using the commit statement.

```
BEGIN;

---- Update post's title and content

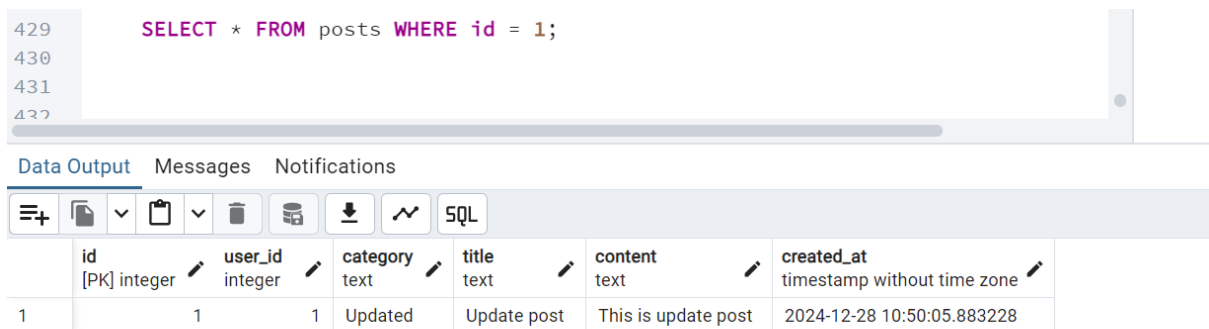
UPDATE posts
SET title='Update post',
content = 'This is update post',
category = 'Updated' WHERE id = 1;

COMMIT;

SELECT * FROM posts WHERE id = 1;
```

Figure 137 2.8.2 Update the posts table

Result:



```
429      SELECT * FROM posts WHERE id = 1;
430
431
432
```

Data Output Messages Notifications						
	id [PK] integer	user_id integer	category text	title text	content text	created_at timestamp without time zone
1	1	1	Updated	Update post	This is update post	2024-12-28 10:50:05.883228

Figure 138 2.8.2 Result in the update posts table

2.9 Summary

In this chapter explore key database management concepts, including indexing, queries, triggers, functions and procedures. Indexing optimizing data retrieval by creating efficient lookup structures, while queries form the foundation for interacting with data through operations like filtering, joining. Transactions ensure data consistency and reliability by grouping multiple operations into a single, automatic unit. Functions and procedures encapsulate reusable logic. Triggers automate action in response to database events, maintaining data integrity.

Chapter 3: CONCLUSION

This project successfully demonstrates the design and implementation of a interactive-driven system. With the integration of tables, functions, procedures, triggers and transactions, it ensures data consistency, integrity and efficiency. The system enables seamless management of posts, comments, likes and notifications offering a structured platforms for user interactions.

By utilizing PostgreSQL features like indexing, foreign key constraints and cascading actions, the project highlights scalable and maintainable database design practices. Furthermore, the use of transactions ensures the atomicity of critical operations, enhancing the system's reliability.

REFERENCES

<https://chatgpt.com/>

<https://www.youtube.com/>